Quantum Spiking Neural Network for Network Intrusion Detection

1st Dr. Parthiban Krishnamoorthy
School of Computer Science Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India
k.parthiban@vit.ac.in

2nd Hiemal Rana
School of Computer Science Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India
hiemal.rana2023@vitstudent.ac.in

3rd Piyush Raj
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India
piyushraj291bd@gmail.com

4th Jayati Jain
School of Computer Science Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India
jayatijain269@gmail.com

***Abstract*—Network intrusion detection systems (NIDS) have to classify large volumes of traffic quickly while keeping false alarms low. Quantum machine learning (QML) and spiking neural networks (SNNs) have both been suggested for building small, cheap detectors, yet in intrusion detection they have only been studied on their own. This paper presents QSNN-NIDS, a hybrid quantum spiking neural network in which every neuron is a quantum leaky integrate-and-fire (QLIF) unit: a parameterized quantum circuit produces the input current, and a classical leaky membrane turns that current into binary spikes over time. Flow records are encoded as phase-coded spike trains over 12 time steps and processed by two QLIF layers of 8 qubits each, with a strongly entangling ansatz, a fast-sigmoid surrogate gradient and a focal loss. We evaluate the model on the UNSW-NB15 benchmark under a leakage-aware protocol with class balancing, mutual-information feature selection and PCA to 8 components. On a held-out test set of 7,284 flows, the model reaches 85.85% accuracy, 89.81% attack recall, an F1 score of 0.841, an MCC of 0.719 and a ROC-AUC of 0.940, using only 340 trainable parameters (96 of them quantum). Neurons fire on 28.4% of time steps, so 71.6% of potential spike events are skipped. Classical baselines trained on the same features still do better; Random Forest reaches 93.34% accuracy. Quantum spiking networks can therefore be trained for this task with very few parameters, but they do not yet match classical accuracy. The results give a reproducible starting point for work on noisy and real quantum hardware.**

***Index Terms*—Quantum machine learning, spiking neural networks, network intrusion detection, variational quantum circuits, leaky integrate-and-fire, UNSW-NB15.**

I. Introduction

Networks now carry far more traffic, from far more devices, than manual monitoring can handle. Intrusion detection systems watch this traffic and raise alerts when a flow looks malicious, and machine learning has become the standard way to build them [1]. In practice a detector must catch most attacks without burying analysts in false alarms, and it must keep pace with live traffic. Deep networks are good at the first part. They tend to be large and power hungry, though, and that makes them awkward to run on gateways and edge devices. Two lines of research look at the cost side of this problem. One is quantum machine learning. Variational quantum circuits and quantum kernels map data into very high dimensional Hilbert spaces using only a handful of qubits and parameters [2]–[4]. Recent studies have applied quantum support vector classifiers, quantum convolutional networks and hybrid quantum neural networks to intrusion detection with promising results [5]–[8]. Current devices, however, belong to the noisy intermediate-scale quantum (NISQ) era [9]: circuits must stay shallow, qubit counts are small, and every circuit evaluation is expensive. A quantum detector has to get a lot out of each circuit call. The other is neuromorphic computing. Spiking neural networks pass information as sparse binary events in time instead of dense real-valued activations [10]. Because a neuron only does work when it fires, SNNs can be far more energy efficient on neuromorphic hardware [11], [12], and surrogate gradient methods now let them be trained end to end with backpropagation [13], [14]. Since network traffic already arrives as a stream of events, spike-based processing suits it well. In intrusion detection, the two directions have not yet met. Quantum IDS models usually treat each flow as a static feature vector, and spiking IDS models are fully classical. Early work on quantum spiking neurons [15], [16] suggests the two can be combined, but we found no study that builds and tests a quantum spiking network for network intrusion detection. That is what we do here.

The main contributions are:

- We propose QSNN-NIDS, a hybrid network built from quantum leaky integrate-and-fire (QLIF) neurons, where a strongly entangling variational circuit supplies the input current and a classical membrane with learnable leak produces spikes over time.
- We describe a leakage-aware pipeline for UNSW-NB15, covering encoding, quantile scaling, class balancing,

mutual-information feature selection, PCA, and phase-coded spike encoding.
- We report detection quality with accuracy, precision, recall, F1, false positive rate, MCC, ROC-AUC and PR-AUC, and compare the model against Random Forest, RBF-SVM and MLP baselines trained on exactly the same features and split.
- We report the quantum resource footprint (qubits, depth, two-qubit gates, circuit evaluations), parameter count, spike sparsity and latency, so that future quantum and neuromorphic IDS work can be compared on cost as well as accuracy.

The rest of the paper is organized as follows. Section II reviews related work. Section III describes the dataset and the proposed model. Section IV gives the experimental setup, Section V presents and discusses the results, Section VI covers limitations, and Section VII concludes.

II. RELATED WORK

A. Machine Learning for Intrusion Detection

Classical IDS research has moved from signature matching to statistical and learned anomaly detection, using decision trees, ensembles, support vector machines and deep networks [1]. The choice of benchmark matters. Older sets such as KDD Cup 99 and NSL-KDD contain dated attacks and many duplicate records, which can inflate reported accuracy. UNSW-NB15 was built to address these issues with modern traffic and nine attack families [17], [18], and it is the benchmark we use here.

B. Quantum Machine Learning for Intrusion Detection

Table I summarizes recent QML studies for intrusion detection. Kernel methods are the most common. Kavitha et al. [5] compared quantum support vector and quantum random forest classifiers on UNSW-NB15 for IoT traffic, Kalinin and Krundyshev [19] applied QSVM and QCNN to streaming security data, and Kumar and Swarnkar [7] built QuIDS around a quantum SVM that learns from few examples. Cirillo et al. [8], [20] benchmarked Pegasos-QSVC, variational classifiers and hybrid networks under hardware noise and found kernel methods to be the most robust. Singh et al. [21] reported that quantum kernels separate benign and malicious traffic more cleanly than classical models on CIC-IDS2017. A second group uses circuit-based neural models. Gong et al. [6] proposed a variational quantum convolutional network for UNSW-NB15, Chao et al. [22] introduced a full-rotation QCNN, Amara et al. [23] designed a hybrid QCNN for healthcare IoT, and Wang et al. [24] showed that hybrid quantum convolutional models generalize well to unseen attacks on 5G-NIDD, although training became unstable when too many features were packed into a small circuit. Shaji et al. [25] compared quantum random forest and QCNN on ToN-IoT. For sequential traffic, Kadi et al. [26] and Do et al. [27] built quantum-enhanced LSTMs for DDoS detection and for managing the precision and recall trade-off. Other studies combined QML with blockchain logging [28], hybrid quantum neural networks for IoT [29], outlier analysis for DDoS [30], quantum metaheuristic optimization [31], software-defined networks [32] and noise-aware circuit mapping [33]. A broad survey of learned IDS for vehicular networks is given in [34].

TABLE I: Recent Quantum Machine Learning Studies for Intrusion Detection

Ref.	Model	Dataset	Reported result
[19]	QSVM, QCNN	Custom stream data	98.00%
[22]	Full-rotation QCNN	KDD Cup 99	99.20%
[6]	VQNN-QCNN	UNSW-NB15	94.51%
[30]	QIDS with outlier analysis	Custom DDoS traffic	99.90%
[5]	QSVC, QRF	UNSW-NB15	99.78%
[28]	QASB-IDS (CNN-LSTM, blockchain)	Edge-IIoTset, SWaT, X-IIoTID	99.10%
[24]	Hybrid QCNN variants	5G-NIDD	81.57% (unseen attacks)
[21]	QSVM vs. DNN, RF	CIC-IDS2017	Quantum above classical
[20]	Pegasos-QSVC, VQC, HQNN	ToN-IoT	93.05%
[23]	Hybrid QCNN	BoT-IoT (healthcare)	about 99%
[7]	QSVC (QuIDS)	Edge-IIoTset, ACI IoT	91.10% recall
[31]	QMGOA with DE	NSL-KDD, BoT-IoT, UNSW-NB15	99.89%
[33]	QuantumNetSec	UNSW-NB15, CIC-IDS2017, ToN-IoT	98.16%
[32]	HQNN, QSVM	UNR-IDD	78.24% (multi-class)
[25]	QRF, QCNN	ToN-IoT	95.00%
[29]	Hybrid QNN	BoT-IoT, ACI IoT, KDD	94.90%
[8]	Pegasos-QSVC	ToN-IoT, NSL-KDD	94.60%
[26]	QLSTM-E	CIC-DDoS2019	99.70%
[27]	Hybrid quantum-classical LSTM	CIC-IDS2017	97.43%
[34]	Survey of ML/DL methods	VeReMi, ToN-IoT, others	Survey
Ours	QSNN-NIDS (this work)	UNSW-NB15	85.85%, MCC 0.719

C. Spiking and Quantum Spiking Neurons

The leaky integrate-and-fire neuron accumulates input current into a membrane potential that slowly leaks, and it emits a spike and resets when the potential crosses a threshold [10]. Because the spike function has no useful gradient, modern SNNs are trained with surrogate gradients that replace the

derivative of the step function with a smooth curve [13], [14], [35]. Kristensen et al. [15] designed an artificial spiking quantum neuron built from a parameterized circuit, and Sun et al. [16] used quantum superposition ideas to encode information in spiking networks. Neither study addressed network security.

D. Research Gap

Looking across Table I, none of the quantum IDS models uses spiking dynamics over time. Each flow is a static vector, and every input costs a full circuit pass. Most papers also report accuracy alone, often on older benchmarks; false positive rate and MCC, which matter more to operators, are rarely given. Quantum resource use (depth, entangling gates, circuit calls) is almost never reported next to detection results, so it is hard to tell whether a model would be practical on a real device. We designed QSNN-NIDS and its evaluation with these gaps in mind.

III. METHODOLOGY

Fig. 1 gives an overview of the pipeline. Raw flow records are cleaned and reduced classically, turned into spike trains, passed through two layers of QLIF neurons over T time steps, and read out as a benign or attack decision.

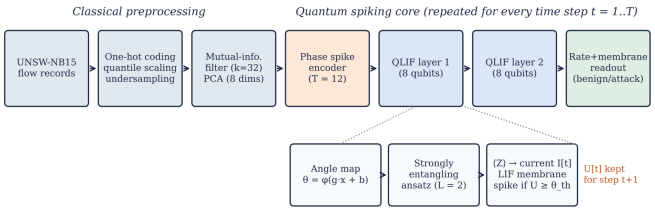


Fig. 1: Overview of QSNN-NIDS. Classical preprocessing reduces each flow to 8 features, which are phase-encoded into spikes and processed by two QLIF layers over T time steps. The lower row shows the computation inside one QLIF layer at a single time step.

A. Problem Formulation

Let each network flow be described by a feature vector x with a label y in 0, 1, where 0 is benign and 1 is attack. The goal is to learn a classifier f(x) that predicts y. A missed attack and a false alarm do not cost the same, so we look at several metrics and not just accuracy.

B. Dataset and Preprocessing

We use the UNSW-NB15 dataset [17], created at the Australian Centre for Cyber Security with the IXIA PerfectStorm tool. It mixes realistic normal traffic with nine attack families (Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode and Worms), described by 49 flow, content, time and connection features. We frame the task as binary detection. From the official training partition (82,332 records) we drew 50,000 records with a fixed seed. Every preprocessing step was fitted on the training split only and then applied as is to the validation and test splits, so no test information reaches training.

Categorical fields (protocol, service and state) were one-hot encoded and identifier and label columns were dropped, leaving 36,419 records with 75 columns. A quantile transformer then mapped every feature to [0, 1], which tames the heavy-tailed byte and packet counts typical of flow data. We split the data with a test size of 0.2 and a validation size of 0.1, and balanced the training split by random undersampling to 10,609 flows per class. The validation and test splits keep their natural class mix.

Mutual information with the label was used to keep the 32 most informative features the top five were sbytes, smean, dbytes and sload. PCA then reduced these to 8 components, which explain 91.5% of the variance. We chose 8 so that each component maps to one qubit.

TABLE II: Data After Preprocessing

Item	Value
Source partition	UNSW-NB15 training set (82,332)
Records drawn	50,000
Records after encoding	36,419 (75 columns)
Features after MI filter	32
PCA components (variance)	8 (91.5%)
Train (balanced)	21,218 (10,609 / 10,609)
Validation	3,642
Test (benign / attack)	7,284 (4,253 / 3,031)

C. Spike Encoding

Each 8-dimensional flow vector is converted into a binary spike tensor $S \in \{0,1\}^{T\times F}$ with T = 12 time steps and F = 8 input channels. We use phase coding, in which the scaled value of a feature controls where in the time window its spikes appear, rather than only how many there are. This carries more information per spike than simple rate coding. The encoder runs in stochastic mode during training, which acts as a light form of data augmentation. The codebase also supports rate, latency, delta and repeat coding; phase coding was selected for the main experiment.

$$s_i[t] = E(x_i, t) \in \{0,1\}, \quad t = 1, \ldots, T \tag{1}$$

D. Quantum Leaky Integrate-and-Fire Neuron

A. Quantum Leaky Integrate-and-Fire Neuron

The core of the model is the QLIF layer, which holds one neuron per qubit. At each time step the layer does four things.

1) Angle mapping. The incoming spike vector produces a drive $z_i[t]$ for each qubit, which is mapped to a bounded rotation angle by a scaled logistic function with gain $g = 2$ and bias $b = -2$:

$$\theta_i[t] = \pi\sigma(g \cdot z_i[t] + b) \tag{2}$$

2) Variational circuit. The angles are loaded into 8 qubits by angle embedding, followed by a strongly entangling ansatz [3]

with $L = 2$ layers. Each layer applies a general single-qubit rotation $\mathrm{Rot}(\alpha, \beta, \gamma)$ to every qubit, then a ring of CNOT gates whose range r grows with the layer index (Fig. 2). With three angles per qubit per layer, each QLIF layer holds $8 \times 2 \times 3 = 48$ quantum weights $\mathbf{w}$.

$$|\psi[t]\rangle = U_{\mathrm{ent}}(\mathbf{w})\, U_{\mathrm{enc}}(\boldsymbol{\theta}[t])\, |0\rangle^{\otimes 8} \tag{3}$$

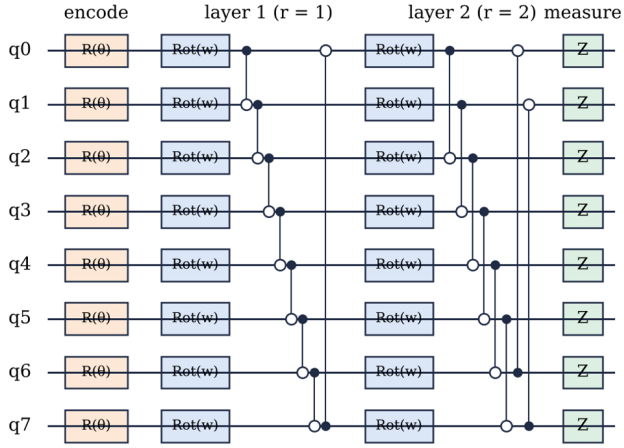


Fig. 2: Circuit used by one QLIF layer at each time step: angle embedding, two strongly entangling layers with CNOT range r = 1 and r = 2, and Pauli-Z measurement on all 8 qubits.

3) Measurement to current. Each qubit is measured in the Pauli-Z basis. The expectation value is converted into a non-negative input current, here written as the probability of observing the excited state:

$$I_i[t] = \frac{1 - \langle \psi[t] | Z_i | \psi[t] \rangle}{2} \tag{4}$$

4) Leaky membrane and spiking. The current is integrated into a membrane potential with leak factor β. A spike is emitted when the potential reaches the threshold V_{th}, and reset by subtraction keeps any excess charge:

$$U_i[t] = \beta U_i[t-1] + I_i[t] - S_i[t-1] V_{\mathrm{th}} \tag{5}$$

$$S_i[t] = H(U_i[t] - V_{\mathrm{th}}) \tag{6}$$

Here H is the Heaviside step function. We set $V_{\mathrm{th}} = 0.55$ and initialize $\beta = 0.9$. The leak is learned through a logit parameter so that it stays between 0 and 1. The membrane gives each neuron a short memory, so the circuit at step t is influenced by everything the neuron has seen before.

E. Surrogate Gradient

The step function in (6) has zero gradient almost everywhere. During the backward pass we replace its derivative with the fast-sigmoid surrogate [35] with slope $\alpha = 5$:

$$\frac{\partial S}{\partial U} \approx \frac{1}{(1 + \alpha |U - V_{\mathrm{th}}|)^2} \tag{7}$$

Gradients flow through the surrogate into the membrane, through the measurement into the circuit, and into both the quantum weights and the learnable leak. On the simulator, gradients are computed exactly by backpropagation through the state vector.

F. Network Architecture and Readout

Fig. 3 shows the full network unrolled over time. Two QLIF layers are stacked, with dropout of 0.1 between them. The second layer runs on the spikes of the first. After T steps, the readout combines the output spike rate, the mean membrane potential and the mean measurement probability of each neuron, and a small linear layer maps these to two class logits:

$$\bar{r}_i = (1/T) \sum_t S_i[t], \quad \hat{y} = \mathrm{softmax}(W_o[\bar{r}; \bar{u}; \bar{p}] + b_o) \tag{8}$$

The full model has 340 trainable parameters: 96 quantum weights (two layers of 48) and 244 classical parameters in the drive, leak and readout. Each flow requires $T \times 2 = 24$ circuit evaluations.

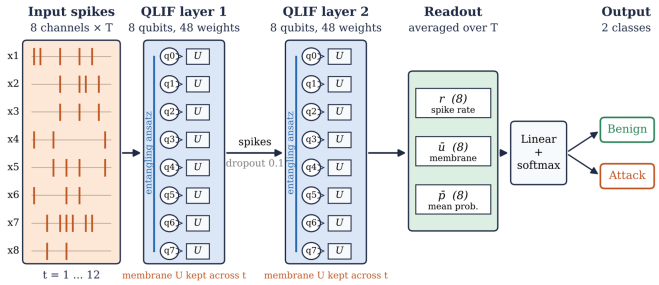


Fig. 3: Network architecture of QSNN-NIDS. Phase-coded input spikes drive two QLIF layers of 8 qubits each; every neuron keeps its membrane potential U from one time step to the next. After T = 12 steps, the readout averages the spike rate, membrane potential and measurement probability of each output neuron, and a linear layer with softmax gives the class.

G. Training Objective

We train with the focal loss [36], which down-weights easy examples so that training concentrates on flows near the decision boundary:

$$\mathcal{L}_{\mathrm{FL}} = -\alpha_c (1 - p_c)^{\gamma} \log(p_c) \tag{9}$$

Here p_c is the predicted probability of the true class and $\gamma = 2$. Because the training set is balanced, class weights α_c are both 1. Optimization uses AdamW [37] with separate learning rates for classical (3×10^{-3}) and quantum (10^{-2}) parameters, weight decay 10^{-4}, a cosine learning-rate

schedule and gradient clipping at norm 1.0. Training stops early if validation F1 does not improve for 8 epochs, and the checkpoint with the best validation F1 is kept.

H. Baselines

To judge the model fairly, we train three classical baselines on exactly the same 8 PCA features and the same train, validation and test split: a Random Forest, an SVM with RBF kernel, and a multilayer perceptron (MLP) trained for 40 epochs. Using identical inputs means any gap in performance comes from the models, not from the features.

I. Evaluation Metrics

With attack as the positive class and TP, TN, FP and FN the confusion matrix counts, we report accuracy, precision, recall (detection rate), F1, false positive rate and the Matthews correlation coefficient:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{10}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{11}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \tag{12}$$

$$\text{MCC} = \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{\sqrt{(\text{TP} + \text{FP})(\text{TP} + \text{FN})(\text{TN} + \text{FP})(\text{TN} + \text{FN})}} \tag{13}$$

We also report balanced accuracy, ROC-AUC and PR-AUC. FPR matters most to operators, since every false positive is an alert someone must check. MCC stays informative when the test classes are unequal in size, as they are here.

IV. EXPERIMENTAL SETUP

All experiments were run in a Kaggle notebook on CPU, using PyTorch 2.10 and PennyLane 0.45.1 [38]. Circuits were simulated exactly, without shot noise or hardware noise. Each training epoch took about 232 seconds, and the full run, including baselines, took about three hours. A fixed random seed of 42 was used for data splitting, encoding and model initialization. Table III lists the main settings.

TABLE III: Model and Training Settings

Setting	**Value**
Spike encoding, time steps T	Phase coding, 12
Qubits per layer, QLIF layers	8, 2
Ansatz, layers L	Strongly entangling, 2
Leak β (initial), learnable	0.9, yes
Threshold Vth, reset	0.55 (fixed), subtract
Angle map gain / bias	2.0 / -2.0
Surrogate, slope α	Fast sigmoid, 5.0
Readout, dropout	Rate + membrane + prob., 0.1
Loss	Focal, $\gamma = 2$
Optimizer, scheduler	AdamW, cosine
Learning rate (classical / quantum)	3×10^{-3} / 10^{-2}
Weight decay, grad. clip	10^{-4}, 1.0
Batch size, max epochs, patience	256, 30, 8
Differentiation, simulator	Backprop, noise-free

TABLE IV: Test Performance of QSNN-NIDS

Metric	**Value**
Accuracy	0.8585
Balanced accuracy	0.8641
Precision (attack)	0.7904
Recall / detection rate (attack)	0.8981
F1 (attack)	0.8408
Macro F1	0.8567
Specificity (benign recall)	0.8302
False positive rate	0.1698
False negative rate	0.1019
MCC	0.7190
ROC-AUC	0.9397
PR-AUC	0.9125

V. RESULTS AND DISCUSSION

A. Detection Performance

Table IV gives the test results for QSNN-NIDS on 7,284 held-out flows (4,253 benign and 3,031 attack). The model reaches 85.85% accuracy and 86.41% balanced accuracy. Its attack recall of 89.81% means it catches about nine of every ten attacks, and the ROC-AUC of 0.940 and PR-AUC of 0.913 show that its scores rank attacks above benign traffic well across thresholds. An MCC of 0.719 indicates that the result holds on both classes rather than leaning on one.

Fig. 4 shows the confusion matrix. Of 3,031 attacks, 2,722 were detected and 309 missed. Of 4,253 benign flows, 3,531 were correctly passed and 722 were flagged, giving a false positive rate of 16.98%. The model leans toward recall and would rather raise a false alarm than miss an attack. Benign precision is high (0.920), so a flow labelled benign is very likely benign, while attack precision is lower (0.790). That makes it better suited to a first-stage filter, with a second and more expensive stage checking its alerts.

B. Training Dynamics

Fig. 5 tracks loss and validation metrics. Training loss fell from 0.147 to about 0.083, and validation loss followed it closely, which points to little overfitting. Validation ROC-AUC climbed from 0.903 to a peak of 0.947 at epoch 21. Validation F1 peaked at 0.847 in epoch 15, after which no improvement held for eight epochs, so training stopped at epoch 23 and the epoch 15 checkpoint was used. At that point the validation accuracy was 86.57% and validation MCC was 0.731, close to the test values, so the validation set was a fair guide to test performance. Validation accuracy and FPR moved up and down from epoch to epoch (for example, FPR rose to 0.280 at epoch 10), which we attribute to the stochastic spike encoder and the small number of quantum parameters.

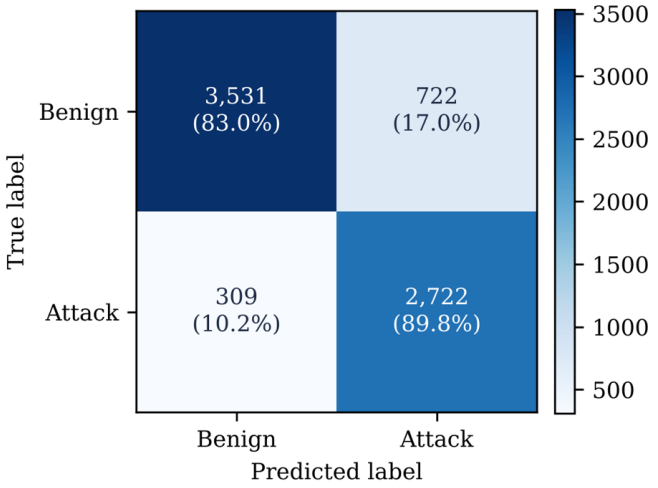


Fig. 4: Confusion matrix on the test set (7,284 flows). Row percentages are shown in brackets.

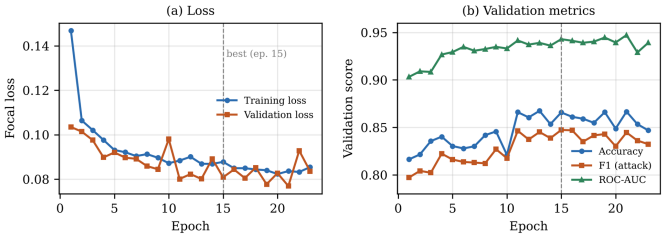


Fig. 5: Training dynamics. (a) Training and validation focal loss. (b) Validation accuracy, F1 and ROC-AUC. The dashed line marks the best checkpoint (epoch 15); early stopping ended training at epoch 23.

C. Comparison with Classical Baselines

Table V and Fig. 6 compare QSNN-NIDS with the classical baselines on identical inputs. All three baselines beat the quantum spiking model. Random Forest does best, with 93.34% accuracy, F1 of 0.922, FPR of 7.17% and ROC-AUC of 0.986. The RBF-SVM and MLP are not far behind it. The largest gap is in false positive rate, where QSNN-NIDS is about 9.8 points behind Random Forest; the smallest is in recall, where it is about 4.3 points behind.

TABLE V: Comparison With Classical Baselines on the Same Features and Split

Model	Acc.	Prec.	Rec.	F1	FPR	AUC
QSNN-NIDS	0.8585	0.7904	0.8981	0.8408	0.1698	0.9397
MLP (DNN)	0.9099	0.8613	0.9340	0.8962	0.1072	0.9750
RBF-SVM	0.9279	0.8933	0.9390	0.9156	0.0799	0.9776
Random Forest	**0.9334**	**0.9034**	**0.9406**	**0.9216**	**0.0717**	**0.9856**

We think it is important to compare against baselines that see exactly the same inputs, since several quantum IDS papers

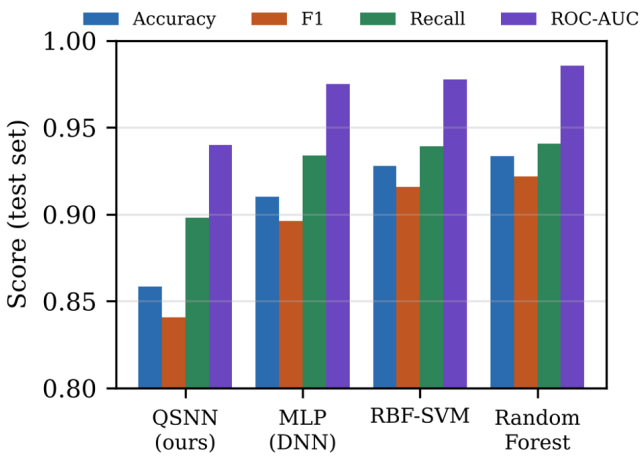


Fig. 6: Test-set comparison of QSNN-NIDS with classical baselines trained on identical inputs.

use weaker or differently processed baselines. Four things likely contribute to the gap. The quantum part of the model has only 96 weights, so its capacity is small next to an ensemble of decision trees. The classical baselines see each flow once as a full vector, while the QSNN must pass information through binary spikes, which loses some precision. The stochastic encoder adds noise to every forward pass. Finally, the firing threshold was fixed at 0.55 rather than learned. Each of these can be changed in future runs. The literature values in Table I are much higher, often above 98%. Those numbers come from different datasets, different splits and, in some cases, protocols without class balancing or with duplicated records, so they cannot be compared directly with ours. Our protocol is stricter: the training set is balanced, the test set is not, all preprocessing is fitted on training data, and the baselines share our features.

D. Quantum Resource Use and Efficiency

Table VI lists the cost of the model. Each QLIF layer uses 8 qubits and a circuit of depth 15 per time step. Across both layers, one time step uses 32 two-qubit gates, and one flow needs 24 circuit evaluations. The circuits are narrow and shallow, small enough to run on current superconducting and trapped-ion devices.

Spiking also makes the model sparse. On the test set, neurons fired on only 28.4% of time steps, so 71.6% of potential spike events did not happen. In a neuromorphic implementation, a silent neuron does no downstream work, so this sparsity translates into fewer operations. Fig. 7 shows that the spike rate fell during training, from 0.296 in epoch 1 to about 0.24 by the end. So the network learned to fire less as its detection improved, even though the loss contains no sparsity penalty.

The whole model has 340 parameters, small enough to store in a few kilobytes; the saved checkpoint is under 9 KB. On the CPU simulator, inference took 3.55 to 3.79 ms per flow, or

TABLE VI: Quantum Resource Footprint and Efficiency

Quantity	Value
Qubits per QLIF layer	8
Circuit depth per time step	15
Total circuit depth (T = 12, 2 layers)	360
Two-qubit gates per time step	32
Circuit evaluations per flow	24
Quantum / classical / total parameters	96 / 244 / 340
Mean spike rate (test)	0.284
Spike sparsity saving	71.6%
Latency per flow (CPU simulator)	3.55 to 3.79 ms
Throughput (CPU simulator)	264 to 282 flows/s

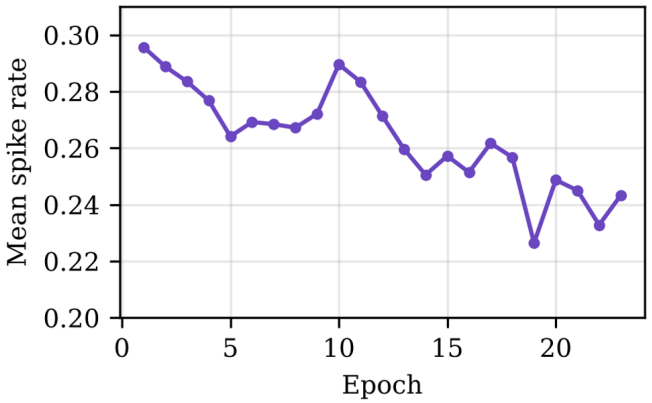


Fig. 7: Mean spike rate of QSNN-NIDS on the validation set across training epochs.

264 to 282 flows per second, at batch size 256. This number is the cost of simulating quantum states on a classical machine. It says nothing about the speed of real quantum hardware and is given only as a reference point.

Discussion

Quantum spiking computation can be trained end to end for intrusion detection. Gradients reach the circuit weights through the surrogate, the model converges in about 15 epochs, and it reaches a ROC-AUC of 0.94. The model is also small, with a few hundred parameters, an 8-qubit circuit and sparse activity, which makes it a candidate for future quantum or neuromorphic edge devices. On accuracy, however, it is not yet competitive with classical models. For now its appeal is its size and sparsity; Random Forest remains the better detector.

VI. LIMITATIONS AND THREATS TO VALIDITY

This study has several limits. All circuits were simulated without noise. The codebase supports depolarizing and amplitude-damping channels, and these were checked in smoke tests, but full noisy runs and real-hardware runs were not part of this experiment, so robustness to NISQ noise remains untested. The results come from a single seed and a single split; repeated runs would give confidence intervals. We used 50,000 records from the official UNSW-NB15 training partition and did not evaluate on the official test partition or on other datasets such as CIC-IDS2017 or ToN-IoT. The task is binary, so performance per attack family is unknown. Quantum baselines (a ZZ-kernel QSVC and a QCNN) are implemented in the same codebase but were only exercised on synthetic smoke-test data, so they are not reported here. Finally, the reported latency is simulator latency on a CPU.

VII. CONCLUSION AND FUTURE WORK

We presented QSNN-NIDS, a quantum spiking neural network for network intrusion detection built from quantum leaky integrate-and-fire neurons. Each neuron uses an 8-qubit strongly entangling circuit to drive a leaky membrane, and the network processes phase-coded flow features over 12 time steps. On UNSW-NB15, with all preprocessing fitted on training data, the model achieved 85.85% accuracy, 89.81% attack recall, an MCC of 0.719 and a ROC-AUC of 0.940 with only 340 parameters and 71.6% spike sparsity. Classical baselines trained on the same features were still more accurate.

Next, we plan to (i) run the model under realistic noise models and on IBM quantum hardware, (ii) learn the firing threshold and study other encodings, (iii) extend the task to multi-class detection of individual attack families, (iv) evaluate on CIC-IDS2017 and ToN-IoT with repeated seeds, (v) complete the comparison with QSVC and QCNN at full scale, and (vi) explore deployment on neuromorphic hardware to measure real energy savings.

CODE AVAILABILITY

The full experiment, including preprocessing, training logs, configuration, and results, is available as a public Kaggle notebook at https://www.kaggle.com/code/hiemalrana22/qml-notebook.

REFERENCES

[1] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: techniques, datasets and challenges," Cybersecurity, vol. 2, Art. no. 20, 2019.

[2] V. Havlíček *et al.*, "Supervised learning with quantum-enhanced feature spaces," Nature, vol. 567, pp. 209–212, 2019.

[3] M. Schuld, A. Bocharov, K. M. Svore, and N. Wiebe, "Circuit-centric quantum classifiers," Phys. Rev. A, vol. 101, no. 3, Art. no. 032308, 2020.

[4] M. Cerezo *et al.*, "Variational quantum algorithms," Nat. Rev. Phys., vol. 3, pp. 625–644, 2021.

[5] S. S. Kavitha, M. D. Dhanush Sagar, and S. Deepak, "Internet of Things network intrusion detection system using quantum and classical machine learning," IET Quantum Commun., vol. 6, no. 1, Art. no. e70020, 2025, doi: 10.1049/qtc2.70020.

[6] C. Gong, W. Guan, H. Zhu, A. Gani, and H. Qi, "Network intrusion detection based on variational quantum convolution neural network," J. Supercomput., vol. 80, no. 9, pp. 12743–12770, 2024, doi: 10.1007/s11227-024-05919-y.

[7] R. Kumar and M. Swarnkar, "QuIDS: A quantum support vector machine-based intrusion detection system for IoT networks," J. Netw. Comput. Appl., vol. 234, Art. no. 104072, 2025, doi: 10.1016/j.jnca.2024.104072.

[8] F. Cirillo, C. Esposito, and J. T. Seo, "Benchmarking quantum machine learning methods for intrusion detection on noisy quantum computers," Quantum Mach. Intell., vol. 8, no. 1, Art. no. 27, 2026, doi: 10.1007/s42484-026-00379-4.

[9] J. Preskill, "Quantum computing in the NISQ era and beyond," Quantum, vol. 2, p. 79, 2018.

[10] W. Maass, "Networks of spiking neurons: The third generation of neural network models," Neural Netw., vol. 10, no. 9, pp. 1659–1671, 1997.

[11] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," Nature, vol. 575, pp. 607–617, 2019.

[12] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," IEEE Micro, vol. 38, no. 1, pp. 82–99, 2018.

[13] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," IEEE Signal Process. Mag., vol. 36, no. 6, pp. 51–63, 2019.

[14] J. K. Eshraghian *et al.*, "Training spiking neural networks using lessons from deep learning," Proc. IEEE, vol. 111, no. 9, pp. 1016–1054, 2023.

[15] L. B. Kristensen, M. Degroote, P. Wittek, A. Aspuru-Guzik, and N. T. Zinner, "An artificial spiking quantum neuron," npj Quantum Inf., vol. 7, Art. no. 59, 2021.

[16] Y. Sun, Y. Zeng, and T. Zhang, "Quantum superposition inspired spiking neural network," iScience, vol. 24, no. 8, Art. no. 102880, 2021.

[17] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS), 2015, pp. 1–6.

[18] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," Inf. Secur. J.: A Global Perspective, vol. 25, no. 1–3, pp. 18–31, 2016.

[19] M. Kalinin and V. Krundyshev, "Security intrusion detection using quantum machine learning techniques," J. Comput. Virol. Hacking Tech., vol. 19, no. 1, pp. 125–136, 2023, doi: 10.1007/s11416-022-00435-0.

[20] F. Cirillo and C. Esposito, "Quantum machine learning for intrusion detection on noisy quantum computers," in Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE), 2025, pp. 1665–1675, doi: 10.1109/QCE65121.2025.00184.

[21] B. Singh, S. Indu, and S. Majumdar, "Comparative analysis of intrusion detection models using quantum machine learning techniques," Circuits Syst. Signal Process., vol. 44, no. 12, pp. 9592–9618, 2025, doi: 10.1007/s00034-025-03256-w.

[22] S. Chao, G. Yang, M. Nie, Y. Liu, and M. Zhang, "Full-rotation quantum convolutional neural network for abnormal intrusion detection system," in Proc. 5th Int. Conf. Artif. Intell. Pattern Recognit. (AIPR), 2022, pp. 852–859, doi: 10.1145/3573942.3574105.

[23] M. Amara, M. Amara, S. Mnasri, and T. Val, "QCNN-ID: A quantum-classical hybrid model for IoT intrusion detection," Procedia Comput. Sci., vol. 270, pp. 2196–2204, 2025, doi: 10.1016/j.procs.2025.09.340.

[24] Z. Wang, K. W. Fok, and V. L. L. Thing, "Network attack traffic detection with hybrid quantum-enhanced convolution neural network," Quantum Mach. Intell., vol. 7, no. 1, Art. no. 50, 2025, doi: 10.1007/s42484-025-00278-0.

[25] A. A. Shaji, S. Sadhwani, R. Muthalagu, P. M. Pawar, and T. Mathew, "QML-IDS: Quantum machine learning approaches for intrusion detection systems in IoT devices," in Proc. 3rd Int. Conf. Comput. Intell. Netw. Syst. (CINS), 2025, pp. 1–7, doi: 10.1109/CINS67018.2025.11412140.

[26] A. Kadi, H. Moudoud, L. Khoukhi, and Z. Abou El Houda, "Quantum-enhanced LSTM for sequential network flow analysis: A hybrid approach to DDoS detection," in Proc. Int. Conf. Quantum Commun., Netw., Comput. (QCNC), 2026, pp. 830–834, doi: 10.1109/QCNC69040.2026.00136.

[27] P. H. Do, T. D. Le, T. D. Dinh, and V.-D. Pham, "Navigating the precision-recall trade-off in network intrusion detection with a hybrid quantum-classical LSTM," in Proc. 40th Int. Conf. Inf. Netw. (ICOIN), 2026, pp. 1006–1011, doi: 10.1109/ICOIN68469.2026.11480537.

[28] N. Hussain, S. Li, A. Hussain, Z. Ullah, and M. Jamjoom, "Quantum-aware secure blockchain intrusion detection system for industrial IoT networks," Sci. Rep., vol. 16, Art. no. 2265, 2026, doi: 10.1038/s41598-025-31985-0.

[29] I. Bharathi, V. Sonai, and S. Sridevi, "Quantum-driven enhanced machine learning algorithm for intrusion detection in Internet of things environment," EPJ Quantum Technol., vol. 13, no. 1, Art. no. 20, 2026, doi: 10.1140/epjqt/s40507-026-00463-5.

[30] T. H. Kim and S. Madhavi, "Quantum intrusion detection system using outlier analysis," Sci. Rep., vol. 14, Art. no. 27114, 2024, doi: 10.1038/s41598-024-78389-0.

[31] A. K. Shukla, "Safeguarding networks from malicious intrusions using quantum machine intelligence," Softw. Pract. Exp., vol. 55, no. 10, pp. 1676–1695, 2025, doi: 10.1002/spe.70004.

[32] J. Lo Anguera *et al.*, "Leveraging quantum machine learning for intrusion detection in software-defined networks," in Proc. IEEE Int. Conf. Mach. Learn. Commun. Netw. (ICMLCN), 2025, pp. 1–6, doi: 10.1109/ICMLCN64995.2025.11140473.

[33] D. Abreu, D. Moura, C. E. Rothenberg, and A. Abelém, "Quantum-NetSec: Quantum machine learning for network security," Int. J. Netw. Manag., vol. 35, no. 4, Art. no. e70018, 2025, doi: 10.1002/nem.70018.

[34] I. Chouchene, H. Eltaief, and H. Youssef, "Advances in machine and deep learning for intrusion detection in VANETs: a comprehensive survey," Cluster Comput., vol. 29, no. 3, Art. no. 143, 2026, doi: 10.1007/s10586-026-05987-0.

[35] F. Zenke and S. Ganguli, "SuperSpike: Supervised learning in multilayer spiking neural networks," Neural Comput., vol. 30, no. 6, pp. 1514–1541, 2018.

[36] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 2980–2988.

[37] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019.

[38] V. Bergholm *et al.*, "PennyLane: Automatic differentiation of hybrid quantum-classical computations," arXiv:1811.04968, 2018.